```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BST:
    def __init__(self):
        self.root = None

    # Insert a value
    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)
        else:
            print("Duplicate values are not allowed.")

        return root

    # Search for a value
    def search(self, root, key):
        if root is None:
            return False

        if root.data == key:
            return True
        elif key < root.data:
            return self.search(root.left, key)
        else:
            return self.search(root.right, key)

    # Find the smallest value in a subtree
    def min_value_node(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current

    # Delete a value
    def delete(self, root, key):
        if root is None:
            return None
```

```
        if key < root.data:
            root.left = self.delete(root.left, key)
        elif key > root.data:
            root.right = self.delete(root.right, key)
        else:
            # Case 1: No left child
            if root.left is None:
                return root.right

            # Case 2: No right child
            if root.right is None:
                return root.left

            # Case 3: Two children
            successor = self.min_value_node(root.right)
            root.data = successor.data
            root.right = self.delete(root.right, successor.data)

        return root

    # Display using inorder traversal
    def display(self, root):
        if root is not None:
            self.display(root.left)
            print(root.data, end=" ")
            self.display(root.right)


# Main program
tree = BST()

while True:
    print("\n\n--- BINARY SEARCH TREE ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Display")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        value = int(input("Enter value to insert: "))
        tree.root = tree.insert(tree.root, value)
        print("Insertion operation completed.")

    elif choice == "2":
        value = int(input("Enter value to delete: "))
        if tree.search(tree.root, value):
            tree.root = tree.delete(tree.root, value)
```

```
            print("Value deleted successfully.")
        else:
            print("Value not found.")

    elif choice == "3":
        value = int(input("Enter value to search: "))
        if tree.search(tree.root, value):
            print("Value found in BST.")
        else:
            print("Value not found.")

    elif choice == "4":
        if tree.root is None:
            print("Tree is empty.")
        else:
            print("BST values in sorted order:")
            tree.display(tree.root)

    elif choice == "5":
        print("Program ended.")
        break

    else:
        print("Invalid choice. Please try again.")
```